

Lecture 04: Pattern Printing & Problem Solving

Is lecture mein hum **nested loops** ka use karke alag-alag **pattern printing** seekhenge, aur saath mein **type conversion** ke concepts ko deeply samajhenge. Ye dono topics interviews mein bahut commonly puche jaate hain.



Part 1: Pattern Printing

INTUITION

Har pattern ko solve karne ka ek hi mantra hai — **outer loop rows ko control karta hai** aur **inner loop columns ko control karta hai**. Pattern ko pehle paper pe draw karo, phir logic identify karo, aur last mein code likho.

Pattern 1: Square Pattern

```
*****  
*****  
*****  
*****  
*****
```

WORKFLOW

- Outer loop 5 baar chalega (rows ke liye)
- Inner loop har row mein 5 stars print karega
- Har row ke baad `endl` se next line

```
// Print 5x5 square of stars
for(int i = 1; i ≤ 5; i++) {
    for(int j = 1; j ≤ 5; j++) {
        cout << "*";
    }
    cout << endl;
}
```

Pattern 2: Right Triangle (Increasing)

```
*
**
***
****
*****
```

INTUITION

Row 1 mein 1 star, Row 2 mein 2 stars... matlab **inner loop ki condition $j \leq i$** honi chahiye. Jab outer loop ka i badhta hai, inner loop utni hi baar chalta hai.

```
for(int i = 1; i ≤ 5; i++) {
    for(int j = 1; j ≤ i; j++) {
        cout << "*";
    }
    cout << endl;
}
```

Pattern 3: Right Triangle (Decreasing)

```
*****
****
***
**
*
```

INTUITION

Isme outer loop **reverse chalta hai** — $i = 5$ se $i = 1$ tak. Har row mein i ke equal stars print hote hain, isliye stars kam hote jaate hain.

```
for(int i = 5; i ≥ 1; i--) {
    for(int j = 1; j ≤ i; j++) {
        cout << "*";
    }
    cout << endl;
}
```

Pattern 4: Inverted Right Triangle

```
  *
 **
***
****
*****
```

WORKFLOW

- **Step 1:** Spaces print karo — $5 - i$ spaces har row mein
- **Step 2:** Stars print karo — i stars har row mein
- **Step 3:** Next line

```
for(int i = 1; i ≤ 5; i++) {
    // Print spaces
    for(int j = 1; j ≤ 5-i; j++) {
        cout << " ";
    }
    // Print stars
    for(int j = 1; j ≤ i; j++) {
        cout << "*";
    }
    cout << endl;
}
```

Key Insight: *Spaces aur stars ka combination* hi pyramid aur diamond patterns ka core logic hai. Spaces ko pehle handle karo, phir stars.

Pattern 5: Pyramid

```
  *
 ***
*****
*****
*****
```

INTUITION

Pyramid mein **odd number of stars** har row mein hote hain: 1, 3, 5, 7, 9. Formula: $2*i - 1$. Spaces ka formula same hai: $n - i$.

```

int n = 5;

for(int i = 1; i ≤ n; i++) {
    // Print spaces
    for(int j = 1; j ≤ n-i; j++) {
        cout << " ";
    }
    // Print stars
    for(int j = 1; j ≤ 2*i-1; j++) {
        cout << "*";
    }
    cout << endl;
}

```

Pattern 6: Inverted Pyramid

```

*****
*****
*****
***
*

```

```

int n = 5;

for(int i = n; i ≥ 1; i--) {
    // Print spaces
    for(int j = 1; j ≤ n-i; j++) {
        cout << " ";
    }
    // Print stars
    for(int j = 1; j ≤ 2*i-1; j++) {
        cout << "*";
    }
    cout << endl;
}

```

Pattern 7: Diamond

```

  *
 ***
*****
*****
*****
 *****
  ***
   *

```

WORKFLOW

- **Upper Half:** Pyramid wala logic (1 se n tak)
- **Lower Half:** Inverted pyramid wala logic (n-1 se 1 tak)
- Diamond = Pyramid + Inverted Pyramid (without repeating middle row)

```
int n = 5;

// Upper half (pyramid)
for(int i = 1; i ≤ n; i++) {
    for(int j = 1; j ≤ n-i; j++) {
        cout << " ";
    }
    for(int j = 1; j ≤ 2*i-1; j++) {
        cout << "*";
    }
    cout << endl;
}

// Lower half (inverted pyramid)
for(int i = n-1; i ≥ 1; i--) {
    for(int j = 1; j ≤ n-i; j++) {
        cout << " ";
    }
    for(int j = 1; j ≤ 2*i-1; j++) {
        cout << "*";
    }
    cout << endl;
}
```

Pattern 8: Hollow Square

```
*****
*   *
*   *
*   *
*****
```

INTUITION

Star tabhi print hota hai jab **first row, last row, first column, ya last column** ho. Baaki jagah space print karo. Condition: `i == 1 || i == n || j == 1 || j == n`.

```
int n = 5;

for(int i = 1; i ≤ n; i++) {
    for(int j = 1; j ≤ n; j++) {
        if(i = 1 || i = n || j = 1 || j = n) {
            cout << "*";
        } else {
            cout << " ";
        }
    }
    cout << endl;
}
```



Pattern 9: Number Triangle

```
1
12
123
1234
12345
```

INTUITION

Har row mein numbers 1 se i tak print hote hain. Inner loop mein j ko directly print karo.

```
for(int i = 1; i ≤ 5; i++) {
    for(int j = 1; j ≤ i; j++) {
        cout << j;
    }
    cout << endl;
}
```

Pattern 10: Number Triangle (Same Number)

```
1
22
333
4444
55555
```

```
for(int i = 1; i ≤ 5; i++) {
    for(int j = 1; j ≤ i; j++) {
        cout << i;
    }
    cout << endl;
}
```

Pattern 11: Number Triangle (Continuous)

```
1
23
456
78910
```

INTUITION

Isme ek **global counter variable** chahiye jo har print ke baad increment ho. num variable ko outer loop ke bahar initialize karo.

```
int num = 1;

for(int i = 1; i ≤ 4; i++) {
    for(int j = 1; j ≤ i; j++) {
        cout << num;
        num++;
    }
    cout << endl;
}
```

Pattern 12: Floyd's Triangle

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

BEST PRACTICE

Floyd's Triangle mein numbers ke beech **space** dena zaroori hai taaki alignment sahi rahe. `cout << num << " ";` ka use karo.

```
int num = 1;

for(int i = 1; i ≤ 5; i++) {
    for(int j = 1; j ≤ i; j++) {
        cout << num << " ";
        num++;
    }
    cout << endl;
}
```

Pattern 13: Binary Triangle

```
1
01
101
0101
10101
```

INTUITION

Pattern observe karo: jab **(i + j) even hota hai** toh 1 print hota hai, warna 0. Ye sum ki parity check hai.

```
for(int i = 1; i ≤ 5; i++) {
    for(int j = 1; j ≤ i; j++) {
        if((i + j) % 2 == 0) {
            cout << "1";
        } else {
            cout << "0";
        }
    }
    cout << endl;
}
```

Pattern 14: Alphabet Triangle

```
A
AB
ABC
ABCD
ABCDE
```



```
for(int i = 1; i ≤ 5; i++) {
    char ch = 'A';
    for(int j = 1; j ≤ i; j++) {
        cout << ch;
        ch++;
    }
    cout << endl;
}
```

Pattern 15: Alphabet Triangle (Repeated)

```
A
BB
CCC
DDDD
EEEE
```

```
char ch = 'A';

for(int i = 1; i ≤ 5; i++) {
    for(int j = 1; j ≤ i; j++) {
        cout << ch;
    }
    ch++;
    cout << endl;
}
```

Pattern 16: Palindrome Number Triangle

```
1
121
12321
1234321
123454321
```

WORKFLOW

- **Step 1:** Spaces print karo ($n - i$)
- **Step 2:** Increasing numbers print karo (1 se i tak)
- **Step 3:** Decreasing numbers print karo (i-1 se 1 tak)

```

int n = 5;

for(int i = 1; i ≤ n; i++) {
    // Print spaces
    for(int j = 1; j ≤ n-i; j++) {
        cout << " ";
    }
    // Print increasing numbers
    for(int j = 1; j ≤ i; j++) {
        cout << j;
    }
    // Print decreasing numbers
    for(int j = i-1; j ≥ 1; j--) {
        cout << j;
    }
    cout << endl;
}

```

Pattern 17: Butterfly Pattern

```

*           *
**          **
***         ***
****        ****
*****       *****
*****       *****
****        ****
***         ***
**          **
*           *

```

INTUITION

Butterfly ko do parts mein divide karo: **Upper Half** aur **Lower Half**. Har row mein left stars, beech mein spaces, aur right stars hote hain. Spaces ka formula: $2*(n-i)$.

```

int n = 5;

// Upper half
for(int i = 1; i ≤ n; i++) {
    // Left stars
    for(int j = 1; j ≤ i; j++) {
        cout << "*";
    }
    // Spaces
    for(int j = 1; j ≤ 2*(n-i); j++) {
        cout << " ";
    }
    // Right stars
    for(int j = 1; j ≤ i; j++) {
        cout << "*";
    }
    cout << endl;
}

// Lower half
for(int i = n; i ≥ 1; i--) {
    for(int j = 1; j ≤ i; j++) {
        cout << "*";
    }
    for(int j = 1; j ≤ 2*(n-i); j++) {
        cout << " ";
    }
    for(int j = 1; j ≤ i; j++) {
        cout << "*";
    }
    cout << endl;
}

```

Pattern 18: Hollow Diamond

```

  *
 * *
*   *
*     *
*       *
*         *
*           *
*             *
*             *
*           *
*         *
*       *
*     *
*   *
 *

```

INTUITION

Hollow diamond mein sirf **boundary pe stars** hote hain. First aur last position pe star, beech mein space. Inner loop mein $j == 1$ ya $j == 2*i-1$ pe star print karo.

```

int n = 5;

// Upper half
for(int i = 1; i ≤ n; i++) {
    for(int j = 1; j ≤ n-i; j++) {
        cout << " ";
    }
    for(int j = 1; j ≤ 2*i-1; j++) {
        if(j = 1 || j = 2*i-1) {
            cout << "*";
        } else {
            cout << " ";
        }
    }
    cout << endl;
}

// Lower half
for(int i = n-1; i ≥ 1; i--) {
    for(int j = 1; j ≤ n-i; j++) {
        cout << " ";
    }
    for(int j = 1; j ≤ 2*i-1; j++) {
        if(j = 1 || j = 2*i-1) {
            cout << "*";
        } else {
            cout << " ";
        }
    }
    cout << endl;
}

```

Pattern 19: Zig-Zag Pattern

```

  *   *
 * * * *
*   *   *

```

INTUITION

Zig-zag pattern mein **diagonal positions** pe stars hote hain. Mathematical observation: $(i + j) \% 4 == 0$ ya special case $i == 2$ && $j \% 4 == 0$.

```
int n = 9; // columns

for(int i = 1; i ≤ 3; i++) {
    for(int j = 1; j ≤ n; j++) {
        if((i + j) % 4 = 0 || (i = 2 && j % 4 = 0)) {
            cout << "*";
        } else {
            cout << " ";
        }
    }
    cout << endl;
}
```

Pattern 20: Plus Pattern

```

*
*
*****
*
*
```

INTUITION

Plus pattern mein **middle row** aur **middle column** pe stars hote hain. $mid = n/2 + 1$ calculate karke condition lagao.

```
int n = 5;
int mid = n/2 + 1;

for(int i = 1; i ≤ n; i++) {
    for(int j = 1; j ≤ n; j++) {
        if(i = mid || j = mid) {
            cout << "*";
        } else {
            cout << " ";
        }
    }
    cout << endl;
}
```

INTERVIEW TIP

Pattern printing questions mein interviewer **logic samajhna** chahta hai, sirf output nahi. Hamesha pehle approach explain karo: "Outer loop rows ke liye, inner loop columns ke liye, aur spaces/stars ka formula yeh hai..."



Part 2: Type Conversion

Type Conversion ka matlab hai ek data type ko dusre data type mein convert karna. C++ mein do tarike hote hain: **Implicit (Automatic)** aur **Explicit (Manual)**.

Implicit Type Conversion

INTUITION

Compiler automatically chhote type ko bade type mein convert kar deta hai. Jaise int ko double mein daalne pe koi data loss nahi hoti, isliye compiler khud handle kar leta hai.

EXAMPLE 1: INT TO DOUBLE

```
int a = 10;
double b = a; // Automatic conversion

cout << b << endl; // 10.0
```

Is case mein **koi data loss nahi hoti** — 10 as it is 10.0 ban jata hai.

EXAMPLE 2: CHAR TO INT

```
char ch = 'A';
int num = ch; // Automatic conversion

cout << num << endl; // 65 (ASCII value)
```

EXAMPLE 3: INT TO FLOAT

```
int x = 5;
float y = x;

cout << y << endl; // 5.0
```

TYPE HIERARCHY

char → short → int → long → float → double

BEST PRACTICE

Smaller to larger conversion hamesha safe hota hai. Data loss nahi hota, aur compiler implicitly handle kar leta hai.

DANGEROUS IMPLICIT CONVERSION

IMPORTANT

Jab **larger type ko smaller type** mein convert karte hain, toh **data loss ya overflow** ho sakta hai. Ye implicit conversion dangerous hai.

EXAMPLE: DOUBLE TO INT (DATA LOSS)

```
double pi = 3.14159;
int num = pi; // Automatic but LOSES decimal part

cout << num << endl; // 3 (lost 0.14159!)
```

EXAMPLE: LARGE TO SMALL (OVERFLOW)

```
int big = 100000;
short small = big; // May overflow

cout << small << endl; // Unpredictable!
```



Explicit Type Conversion (Type Casting)

Explicit conversion mein programmer manually cast operator ka use karke type change karta hai. C++ mein do tarike hain:

METHOD 1: C-STYLE CAST

```
(type)variable
```

```
double pi = 3.14159;
int num = (int)pi; // Explicit cast

cout << num << endl; // 3
```

METHOD 2: C++ STYLE CAST (RECOMMENDED)

```
static_cast<type>(variable)
```

```
double pi = 3.14159;
int num = static_cast<int>(pi);

cout << num << endl; // 3
```

BEST PRACTICE

Hamesha `static_cast` prefer karo C-style cast ke upar. Ye compile-time check karta hai aur type safety better hoti hai.

WHY USE EXPLICIT CASTING?

EXAMPLE 1: DIVISION PROBLEM

```
int a = 10;
int b = 3;

cout << a / b << endl; // 3 (integer division!)

// Fix: Cast to double
cout << (double)a / b << endl; // 3.33333
```

INTUITION

- **Without cast:** $10 / 3 \rightarrow$ dono int hain $\rightarrow$ result bhi int $\rightarrow 3$
- **With cast:** $(\text{double})10 / 3 \rightarrow 10.0 / 3 \rightarrow$ result double $\rightarrow 3.33333$

EXAMPLE 2: PERCENTAGE CALCULATION

```
int marks = 85;
int total = 100;

// Wrong
int percentage = (marks / total) * 100;
cout << percentage << endl; // 0 (why?)

// Correct
double percentage = ((double)marks / total) * 100;
cout << percentage << endl; // 85
```

EDGE CASE

`85 / 100` integer division mein `0` hota hai, kyunki 85 ko 100 se divide karne pe quotient 0 aata hai. Phir $0 * 100 = 0$. Isliye pehle kisi ek operand ko double mein cast karna zaroori hai.

EXAMPLE 3: AVERAGE CALCULATION


```
int a = 10, b = 20, c = 30;

// Wrong
int avg = (a + b + c) / 3;
cout << avg << endl; // 20 (loses decimal)

// Correct
double avg = (double)(a + b + c) / 3;
cout << avg << endl; // 20.0
```

TYPE CONVERSION IN EXPRESSIONS

Key Insight: Expression ka result hamesha **sabse bade type** ke according hota hai. Agar expression mein double hai, toh final result double hoga.

Expression	Result Type	Example
int + int	int	5 + 3 = 8
int + double	double	10 + 3.5 = 13.5
float + double	double	2.5f + 3.5 = 6.0
char + int	int	'A' + 1 = 66

EDGE CASE

int x = 5; int y = 2; double z = x / y; → **z = 2.0** (NOT 2.5!)

Kyunki x / y pehle int division hota hai ($5/2 = 2$), aur phir wo 2 double mein convert hota hai (2.0). Agar 2.5 chahiye, toh pehle cast karo.



Practice Problems

Problem 1: Temperature Converter

WORKFLOW

Celsius input lo, formula $(\text{celsius} * 9.0 / 5.0) + 32$ se Fahrenheit calculate karo. 9.0 likhna zaroori hai taaki floating-point division ho.

```

#include <iostream>
using namespace std;

int main() {
    double celsius;

    cout << "Enter temperature in Celsius: ";
    cin >> celsius;

    double fahrenheit = (celsius * 9.0 / 5.0) + 32;

    cout << celsius << "°C = " << fahrenheit << "°F" << endl;

    return 0;
}

```

Problem 2: Grade Percentage

```

#include <iostream>
using namespace std;

int main() {
    int subject1, subject2, subject3;

    cout << "Enter marks of 3 subjects: ";
    cin >> subject1 >> subject2 >> subject3;

    int total = subject1 + subject2 + subject3;
    double percentage = (double)total / 3;

    cout << "Total: " << total << endl;
    cout << "Percentage: " << percentage << "%" << endl;

    return 0;
}

```

Problem 3: ASCII Value

```

#include <iostream>
using namespace std;

int main() {
    char ch;

    cout << "Enter a character: ";
    cin >> ch;

    int ascii = (int)ch;

    cout << "ASCII value of " << ch << " is " << ascii << endl;

    return 0;
}

```



Bonus Problems

Prime Number Check

INTUITION

Prime number wo hota hai jo **sirf 1 aur apne aap se divisible** ho. 2 se chhoti value kabhi prime nahi hoti. 2 se lekar num-1 tak koi bhi divisor mil gaya toh not prime.

```
int num;

cout << "Enter the Number: ";
cin >> num;

// 2 se chhoti value kabhi bhi prime nahi hoti.
if (num < 2) {
    cout << num << " is Not a Prime Number";
    return 0;
}

// 2 se num-1 tak divide karke check karo.
for (int i = 2; i < num; i++) {
    if (num % i == 0) {
        cout << num << " is Not a Prime Number";
        return 0;
    }
}

cout << num << " is a Prime Number" << endl;
```

PERFORMANCE

Is approach ki **time complexity $O(n)$** hai. Optimized approach mein sirf $\sqrt{\text{num}}$ tak check karte hain, jo $O(\sqrt{n})$ complexity deta hai. Interview mein optimized version bhi batana.

Sum of Digits

WORKFLOW

- **Step 1:** number % 10 se last digit nikalo
- **Step 2:** Us digit ko sum mein add karo
- **Step 3:** number / 10 se last digit remove karo
- **Step 4:** Jab tak number > 0, repeat karo

```
int number = 12345;
int sum = 0;

while (number > 0) {
    int lastDigit = number % 10; // Last digit nikalo

    sum = sum + lastDigit; // Sum me add karo

    number = number / 10; // Last digit remove karo
}

cout << "Sum of Digits: " << sum << endl;
```



Key Takeaways

TAKEAWAY

- **Pattern Printing:** Outer loop = rows, Inner loop = columns. Spaces aur stars ka formula pehle identify karo.
- **Implicit Conversion:** Compiler automatically smaller → larger type mein convert karta hai. Safe hai, data loss nahi hota.
- **Explicit Conversion:** Programmer manually cast karta hai. `static_cast` prefer karo C-style cast ke upar.
- **Integer Division Trap:** Do integers divide karne pe decimal part lost ho jata hai. Hamesha explicit cast karo jab precise result chahiye.
- **Prime Check:** 2 se $n-1$ tak check karo. Optimized version mein $\sqrt{n}$ tak check karo.
- **Sum of Digits:** Modulo 10 se digit nikalo, divide by 10 se remove karo. Loop until number becomes 0.

Type	When	Example
Implicit	Automatic	<code>int a = 5; double b = a;</code>
Explicit	Manual cast	<code>int x = (int)3.14;</code>
Explicit (C++)	Recommended	<code>int x = static_cast<int>(3.14);</code>

INTERVIEW TIP

Type conversion questions mein interviewer **integer division trap** check karta hai. Hamesha yaad rakho: `int / int = int`. Agar decimal result chahiye, toh pehle kisi ek operand ko `double` mein cast karo. Ye common mistake freshers ki interviews mein hoti hai.